informs
https://pubsonline.informs.org/journal/opre

OPERATIONS RESEARCH

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *Operations Research*

ISSN 0030-364X, EISSN 1526-5463

Robust Candidate MaxWeight Certificates for Heterogeneous Compute Scheduling: A Scheduleurm Case Study

(Authors' names are not included for peer review)

Abstract. Modern compute clusters serve jobs whose rates depend on co-location, host load, accelerator memory, and placement. This makes the natural scheduling action a full configuration rather than a scalar resource allocation. We model the problem as a configuration-action stochastic processing network and prove a robust candidate MaxWeight theorem. The theorem keeps the full action space in the capacity statement and charges four implementable losses against explicit slack: candidate approximation, conservative service estimation, bounded penalties, and approximate optimization. A positive residual margin gives a finite-set Foster drift certificate under bounded conditional second moments. We provide a Lean formalization of the fixed-family and statewise statements. We instantiate the model in Scheduleurm, an operational research-cluster scheduler. The implementation records measured lower-service rows, admissible candidate configurations, penalty terms, oracle gaps, and production-admission status. Across declared finite service slices and 192 Poisson and bursty replay scenarios, the robust candidate improves geometric-mean makespan by 1.61× and mean flow by 4.02× over legacy Scheduleurm caps. On the same measured service cache, it is not Pareto-dominated by the registered policy-semantics baselines used to represent throughput, delay, and interference-control schedulers. The evidence supports a measured finite-slice stability and performance claim, with external full-stack and future-workload generalization left outside the main claim.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

Subject classifications: Queues: scheduling; stochastic models: processing networks; computing: resource allocation

Area of review: Stochastic Models; Computing

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

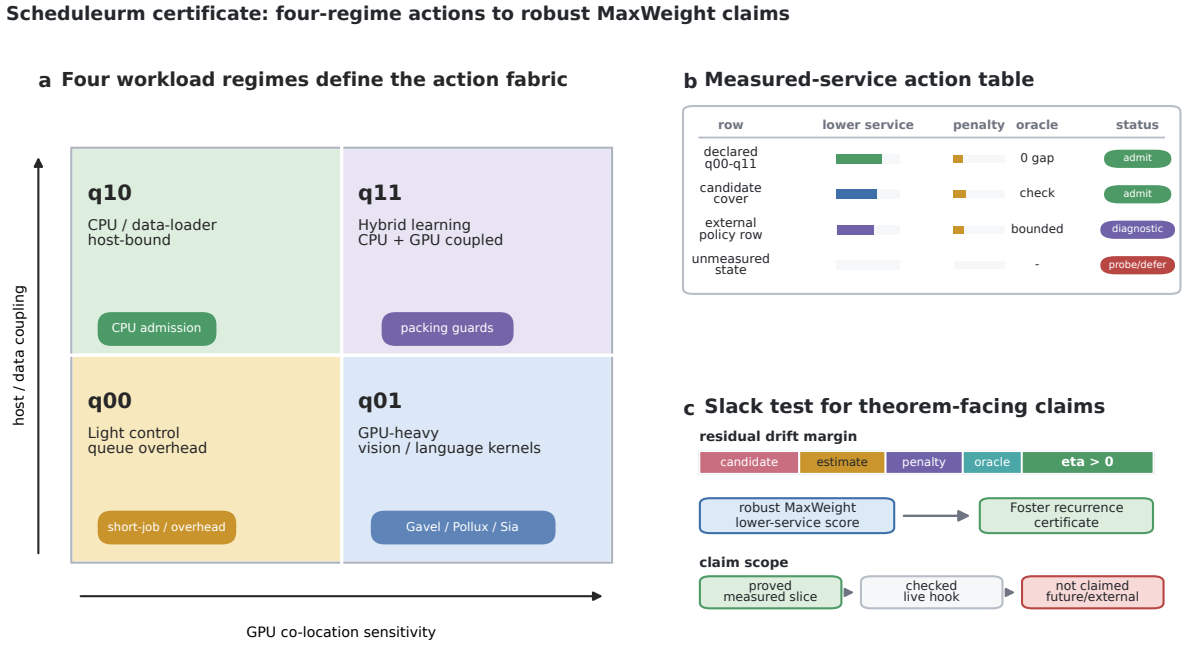

Figure 1 Overview of the robust candidate MaxWeight certificate. A combinatorial full action space is represented by measured finite candidate actions across four workload quadrants. The scheduler scores candidates using the queue-weighted lower-service objective minus bounded penalties, and the stability certificate requires the residual slack to exceed candidate-cover loss, service-estimation error, penalty growth, and approximate-oracle error. External policy families are diagnostic finite actions on the same service cache, not universal full-stack baselines. The measured-service action table summarizes lower-service rows, bounded penalties, oracle gaps, and admission status.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked "sweet spots" can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack. Figure 1 gives the one-page certificate structure: candidate construction, lower-service scoring, slack accounting, and bounded claim boundaries are checked as separate objects.

The main theoretical contribution is a robust candidate MaxWeight theorem with complete slack accounting. The theorem states exactly how candidate approximation, conservative service estimation, scheduling penalties, and approximate optimization consume capacity slack. When the remaining margin is positive and the stochastic primitives have bounded conditional second

moments, the resulting policy satisfies a finite-set Foster recurrence certificate. The result is stated for both fixed feasible families and state-dependent feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and measured capacity boundaries.

The empirical contribution is a theorem-condition evaluation, not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in Scheduleurm, an existing scheduler used for local research workloads, and report bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit replay compares robust candidate actions with legacy rules and diagnostic policy families on a common measured service cache, so all policies are evaluated against the same lower-service object. Third, holdout diagnostics, finite lower-service capacity certificates, ablation, production coverage, and live-state dry-run trace checks identify which assumptions are certified and which remain finite-slice claims. These results connect the proof obligations to scheduler artifacts; they do not replace a universal fabric-cover theorem or an exhaustive full-stack scheduler competition.

The paper is organized as follows. We first state the claim scope and evidence contract, then position the work relative to scheduler and queueing literatures. We then introduce the configuration-action processing network, give the candidate approximation and robust MaxWeight stability theorem, describe the Scheduleurm instantiation, report the computational evidence, and close with the remaining calibration limits.

2. Scope and Contributions

Table 1 states the scope used throughout the paper. Each row connects one claim to the evidence used for it and to the stronger statement that the paper does not assert.

Table 1 Claim scope and evidence.

Claim	Evidence used here	Boundary
Robust candidate MaxWeight stability	Main-text proof, electronic-companion derivation, and Lean checks for exact and approximate oracle versions.	Holds only under the stated slack, lower-service, penalty, oracle, and moment conditions.
Measured finite service domains	Declared q00/q01/q10/q11 service slices, mixed-portfolio replay, lower-service slack certificates, and probe/defer status for unknown states.	Does not imply positive service for unmeasured workloads, hardware states, or arbitrary future arrivals.
Scheduleurm theorem objects	Measured service rows, candidate configurations, admission boundaries, penalty records, oracle-gap checks, and optional live-hook fields.	Does not make every production dispatch a theorem-grade global MaxWeight action.
Controlled and production evidence	Controlled launched/completed batches, strict completed-active production history, read-only shadow traces, and production readiness checks.	Supports admitted rows and controlled launches; raw telemetry, attempted-only rows, and future unknown jobs remain outside the theorem population.
External scheduler comparison	Registered policy-semantics baselines evaluated on the same measured service cache, plus scoped adapter or same-host runtime probes when available.	Does not claim direct full-stack superiority over every external scheduler or original multi-node deployment.
Learning and hidden-regime extensions	Formal conditional results and finite active-bucket replay checks, forced exploration, switching, and detector-delay records.	Treated as extensions rather than main live performance or complete online-learning claims.

The table separates what is proved or measured from nearby claims that require additional experiments or assumptions.

3. Related Work

3.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Learning-based schedulers such as Decima and DL2 show another way to adapt scheduling decisions from traces or reinforcement learning (Mao et al. 2019, Peng et al. 2019). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). Scheduleurm follows the same empirical lesson. Its q00, q10, and q11 buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

3.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004, Neely 2010). Foster-Lyapunov and fluid-limit arguments also underlie positive recurrence and moment control in multiclass queueing networks (Dai and Meyn 1995, Meyn and Tweedie 2009). Our proof follows this lineage but adds three implementation-facing terms: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates. The current experiment artifact closes the deterministic finite active-bucket union bound and replay dwell/switching accounting, and adds a concrete deployable probability model: deterministic round-robin forced exploration over the finite active bucket set and a bounded two-window mean-shift detector. The certificate covers 64 active buckets and 120 replay scenarios with minimum forced samples 1561 per bucket, Hoeffding radius 0.0523, detector union tail bound 0.00564, and detection-delay bound 1024 decisions. A 2026-06-12 optional live-state probe further verifies that the theorem-facing live hook emits active-bucket sampler and regime-detector records while preserving the lower-service theorem oracle. This remains an opt-in extension path; the default live scheduler policy is not changed.

4. Model

4.1. Notation

Table 2 fixes the notation used throughout the paper. Each mathematical symbol is reserved for the meaning shown in the table; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

4.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t. A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served,

Table 2 Notation.

Symbol	Meaning
$\mathcal{I}$	finite set of job classes
i	job-class index
t	discrete decision slot
$Q_i(t)$	class-i backlog at slot t
$Q(t)$	queue vector at slot t
λ	arrival mean vector
v	feasible service vector
a	configuration action
a_t	action selected at slot t
$\mathcal{A}^{\text{full}}$	full configuration-action family
$\mathcal{A}^{\text{cand}}$	fixed candidate-action family
$\mathcal{A}_t^{\text{cand}}$	statewise candidate-action family at slot t
$\mathcal{A}^{\text{meas}}$	measured finite action slice
Z_t	hidden service regime at slot t
$\chi(t)$	scheduler-visible state at slot t
ω_t	exogenous randomness at slot t
$A_i(t)$	class-i arrival at slot t
$S_i(t)$	class-i realized service at slot t
$\mu^z(a)$	service mean of action a in regime z
$\mu(a)$	main-theorem service mean of action a
$\mu_t^{\text{L}}(a)$	conservative lower-service estimate for action a at slot t
$C(\mathcal{A})$	convex hull of service vectors generated by action family $\mathcal{A}$
$\Lambda(\mathcal{A})$	downward-closed load region of action family $\mathcal{A}$
$\text{Mix}(\mathcal{A})$	probability simplex over action family $\mathcal{A}$
x_a	stationary mixing mass assigned to action a
q	nonnegative support vector
$H_{\mathcal{A}}(q)$	support function of action family $\mathcal{A}$ in direction q
Φ	finite feature map for configuration actions
m	feature dimension
d_Φ	weighted fabric metric induced by Φ
w_r	weight of feature coordinate r
L	service Lipschitz envelope in the fabric metric
ρ	candidate-cover radius
π	projection from full actions to candidate actions
$K_t(a)$	switching cost of action a at slot t
$G_t(a)$	risk or interference penalty of action a at slot t
$P_t(a)$	total penalty $K_t(a) + G_t(a)$
P_0	additive penalty bound
β	queue-scaled penalty coefficient
θ_{prof}	dimensionless profile-penalty fraction in replay
δ	full-region slack
ϵ_{cand}	candidate-set support loss
ϵ_{est}	lower-service estimation loss
α_0	additive approximate-oracle error
α_1	queue-scaled approximate-oracle error
η	residual drift margin
B	second-moment bound
N	finite-set threshold
γ	negative-drift margin outside the finite set
V	quadratic Lyapunov function
ΔV	one-step Lyapunov drift
b	replay backlog count
k	measured co-location profile
$\mathcal{K}_i$	measured support envelope of profiles for class i
$k_i^\star(b)$	replay profile selected for class i at backlog count b
$\widehat{\mu}_i(k)$	measured aggregate service of class i at profile k
$\widehat{\mu}_i^{\max}$	best measured aggregate service for class i in the same envelope

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure

buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω, action a generates a nonnegative service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use $\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

The queue dynamics are

$$Q_i(t+1) = [Q_i(t) - S_i(t)]^+ + A_i(t), \qquad i \in \mathcal{I}, \tag{1}$$

where $A_i(t)$ is the class-i arrival in slot t. In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to assume bounded conditional second moments.

4.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \mathrm{Mix}(\mathcal{A}) \right\}, \tag{2}$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \le v \right\}. \tag{3}$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \qquad q \in \mathbb{R}_+^{|\mathcal{I}|}. \tag{4}$$

Here $\mathrm{Mix}(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a. The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \mathrm{Mix}(\mathcal{A}^{\mathrm{full}})$ such that $\lambda_i + \delta \le \sum_a x_a \mu_i(a)$ for every i. Consequently,

$$q^\top \lambda + \delta \, \|q\|_1 \le H_{\mathcal{A}^{\mathrm{full}}}(q), \qquad q \ge 0. \tag{5}$$

Inequality (5) is the bridge from capacity to MaxWeight drift; it is not an operational stability "if and only if" by itself.

4.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\text{full}} \to \mathbb{R}^m$ and weighted ℓ_1 fabric metric

$$d_\Phi(a, a') = \sum_{r=1}^{m} w_r |\Phi_r(a) - \Phi_r(a')|. \tag{6}$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\text{cand}}$ is a ρ-cover of $\mathcal{A}^{\text{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\text{cand}}$ with $d_\Phi(a, \pi(a)) \le \rho$. If $\|\mu(a) - \mu(a')\|_\infty \le L d_\Phi(a, a')$, then

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \, \|q\|_1 , \qquad q \ge 0. \tag{7}$$

The support-loss bound (7) is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ. For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

5. Robust Candidate MaxWeight

5.1. Policy

Let $\mu_t^{\text{L}}(a)$ be a conservative lower-service estimate for candidate action a at time t. Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a, and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$\begin{aligned} a_t \in \arg \max_{a \in \mathcal{A}_t^{\text{cand}}} \big\{ & Q(t)^\top \mu_t^{\text{L}}(a) \\ & - K_t(a) - G_t(a) \big\}. \end{aligned} \tag{8}$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (8),

$$\begin{aligned} & \max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q^\top \mu_t^{\text{L}}(a) - K_t(a) - G_t(a) \right\} \\ & \quad - \left\{ Q^\top \mu_t^{\text{L}}(a_t) - K_t(a_t) - G_t(a_t) \right\} \\ & \le \alpha_0 + \alpha_1 \, \|Q\|_1 . \end{aligned} \tag{9}$$

The oracle loss in (9) is paid from the same slack budget as candidate loss and lower-service error. Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

Table 3 Proof roadmap for Theorem 1.

Stage	Mathematical role	Term consumed
Capacity support	Convert full-action slack into a support-function margin.	None.
Candidate cover	Compare the full action family with the candidate family.	ϵ_{cand}, equal to $L\rho$ when a fabric metric is used.
Lower service	Replace unknown mean service by conservative service rows.	ϵ_{est}.
Penalty and oracle	Account for switching, risk, and approximate optimization.	$P_0, \beta, \alpha_0, \alpha_1$.
Drift margin	Combine the preceding losses into the residual slope η.	Requires $\eta > 0$.
Foster step	Use the quadratic Lyapunov inequality and second-moment bound to obtain negative drift outside a finite set.	B, N, γ and the local-return condition.

The table shows why each assumption is needed. Lean checks the algebraic implications; service, cover, oracle, and moment conditions are supplied by the measured Scheduleurm certificates.

5.2. Main stability certificate

5.2.1. Assumptions We state the assumptions explicitly because each one corresponds to a measurable artifact in Scheduleurm. All constants below are nonnegative and are fixed over the horizon under consideration.

Table 3 is the proof spine used below. The numbered lemmas give the main-text derivation; Appendix "Conventional Proof Details" then expands the same chain in electronic-companion style so that the reader can check the theorem without opening the Lean files.

ASSUMPTION 1 **(Load slack)**. *The arrival mean vector λ has full-action support slack $\delta > 0$:*

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q), \qquad q \in \mathbb{R}_+^{|\mathcal{I}|}. \tag{10}$$

This is the support-function form of belonging to the interior of the full-action capacity base. It is a sufficient stability hypothesis; the operational necessity direction requires a separate conservation law.

ASSUMPTION 2 **(Candidate cover and lower-service error)**. *For every scheduler-visible state t and every $q \in \mathbb{R}_+^{|\mathcal{I}|}$,*

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 , \tag{11}$$

and

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + \epsilon_{\text{est}} \|q\|_1 . \tag{12}$$

When the fabric metric in Section 4.4 is certified, $\epsilon_{\text{cand}} = L\rho$. For an exact measured finite slice, $\epsilon_{\text{cand}} = 0$.

ASSUMPTION 3 **(Penalty growth and approximate oracle)**. *Let* $P_t(a) = K_t(a) + G_t(a)$. *For all* $a \in \mathcal{A}_t^{\text{cand}}$,

$$0 \le P_t(a) \le P_0 + \beta \|Q(t)\|_1 . \tag{13}$$

The selected action $a_t \in \mathcal{A}_t^{\text{cand}}$ *satisfies the approximate-oracle condition*

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q(t)^\top \mu_t^{\text{L}}(a) - P_t(a) \right\} \\ &\quad - \left\{ Q(t)^\top \mu_t^{\text{L}}(a_t) - P_t(a_t) \right\} \le \alpha_0 + \alpha_1 \|Q(t)\|_1 . \end{aligned} \tag{14}$$

Exact robust MaxWeight is the special case $\alpha_0 = \alpha_1 = 0$.

ASSUMPTION 4 **(Stochastic domination and bounded second moments)**. *Conditional on* $Q(t) = Q$,

$$\mathbb{E}[A_i(t) \mid Q(t) = Q] \le \lambda_i, \tag{15}$$

and

$$\mu_{t,i}^{\text{L}}(a_t) \le \mathbb{E}[S_i(t) \mid Q(t) = Q] \tag{16}$$

for every class i, *and*

$$\mathbb{E}\left[\frac{1}{2} \sum_i \{A_i(t)^2 + S_i(t)^2\} \,\middle|\, Q(t) = Q \right] \le B. \tag{17}$$

ASSUMPTION 5 **(Countable-state recurrence condition)**. *For a positive-recurrence conclusion, the induced controlled process is a Markov chain on a countable queue state space, sublevel sets* $\{Q : \|Q\|_1 \le N\}$ *are finite, and the usual local-return condition holds on the finite set selected by the drift certificate. Without this assumption we claim only the finite-set negative-drift certificate.*

5.2.2. Deterministic slack accounting

LEMMA 1 **(Support slack after candidate and estimation losses)**. *Under Assumptions 1–2, for every* t *and* $q \in \mathbb{R}_+^{|\mathcal{I}|}$,

$$\begin{aligned} \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) &\ge q^\top \lambda \\ &\quad + (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}}) \|q\|_1 . \end{aligned} \tag{18}$$

Proof. By Assumption 1,

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q). \tag{19}$$

Assumption 2 gives

$$\begin{aligned} H_{\mathcal{A}^{\text{full}}}(q) &\le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 \\ &\le \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) \\ &\quad + (\epsilon_{\text{cand}} + \epsilon_{\text{est}}) \|q\|_1 . \end{aligned} \tag{20}$$

Rearranging proves the claim.

LEMMA 2 **(Selected lower-service margin)**. *Under Assumptions 1–3, for $Q = Q(t)$,*

$$\begin{aligned} Q^\top \mu_t^{\text{L}}(a_t) \ge Q^\top \lambda & \\ + \left(\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}} - \beta - \alpha_1\right) \|Q\|_1 & \\ - (P_0 + \alpha_0). & \end{aligned} \tag{21}$$

Proof. Let $M(Q) = \max_{a \in \mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^{\text{L}}(a)$. Since $P_t(a) \le P_0 + \beta \|Q\|_1$ for every candidate,

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q^\top \mu_t^{\text{L}}(a) - P_t(a)\} \\ &\quad \ge M(Q) - P_0 - \beta \|Q\|_1 . \end{aligned} \tag{22}$$

The approximate-oracle condition therefore implies

$$\begin{aligned} Q^\top \mu_t^{\text{L}}(a_t) - P_t(a_t) \ge M(Q) - P_0 - \beta \|Q\|_1 & \\ - \alpha_0 - \alpha_1 \|Q\|_1 . & \end{aligned} \tag{23}$$

Because $P_t(a_t) \ge 0$, the left-hand side is at most $Q^\top \mu_t^{\text{L}}(a_t)$. Applying Lemma 1 with $q = Q$ gives the stated inequality.

LEMMA 3 **(Quadratic one-step bound)**. *For $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and nonnegative vectors Q, A, S,*

$$\begin{aligned} V([Q - S]^+ + A) - V(Q) & \\ \le \frac{1}{2} \sum_i (A_i^2 + S_i^2) + Q^\top A - Q^\top S. & \end{aligned} \tag{24}$$

Proof. It is enough to prove the coordinate inequality. For $q, a, s \ge 0$, $([q - s]^+)^2 \le (q - s)^2$ and $[q - s]^+ \le q$. Hence

$$\begin{aligned} ([q - s]^+ + a)^2 &\le ([q - s]^+)^2 + a^2 + 2a[q - s]^+ \\ &\le q^2 + s^2 + a^2 + 2q(a - s). \end{aligned} \tag{25}$$

Subtracting q^2, dividing by two, and summing over coordinates gives the result.

THEOREM 1 (**Robust candidate MaxWeight stability certificate**). *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics* (1)*. Suppose Assumptions 1–4 hold. If*

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1) > 0, \tag{26}$$

then for $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] \le B + P_0 + \alpha_0 \\ - \eta \left\|Q\right\|_1 .\end{aligned} \tag{27}$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \le \eta N \tag{28}$$

for some $\gamma > 0$, the process has a negative expected drift outside $\{Q : \|Q\|_1 \le N\}$. If Assumption 5 also holds, the standard countable-state Foster criterion gives positive recurrence of the corresponding closed communicating class.

Proof. Apply Lemma 3 with $(Q, A, S) = (Q(t), A(t), S(t))$ and condition on $Q(t) = Q$. Using Assumption 4,

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\le B + Q^\top \mathbb{E}[A(t) \mid Q(t) = Q] \\ &\quad - Q^\top \mathbb{E}[S(t) \mid Q(t) = Q] \\ &\le B + Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t).\end{aligned} \tag{29}$$

Write $D(Q) = Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t)$. Lemma 2 gives

$$\begin{aligned}D(Q) \le P_0 + \alpha_0 \\ - \left(\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}} - \beta - \alpha_1\right) \left\|Q\right\|_1 .\end{aligned} \tag{30}$$

Combining the two displays proves

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] \le B + P_0 + \alpha_0 \\ - \eta \left\|Q\right\|_1 .\end{aligned} \tag{31}$$

If $B + P_0 + \alpha_0 + \gamma \le \eta N$, the drift is at most $-\gamma$ whenever $\|Q\|_1 > N$. This is the finite-set Foster drift certificate. Under Assumption 5, the usual countable-state Foster theorem yields positive recurrence of the closed class.

COROLLARY 1 **(Statewise feasible families)**. *Let the candidate family and lower-service map be state indexed:* $\mathcal{A}_t^{\mathrm{cand}} = \mathcal{A}^{\mathrm{cand}}(Q(t), \chi(t))$ *and* $\mu_t^{\mathrm{L}} = \mu^{\mathrm{L}}(Q(t), \chi(t), \cdot)$*, where* $\chi(t)$ *records scheduler-visible availability, pinned jobs, node health, and measured capacity boundaries. If Assumptions 1–4 hold pointwise for every realized* $(Q(t), \chi(t))$ *with the same constants* $\delta, \epsilon_{\mathrm{cand}}, \epsilon_{\mathrm{est}}, P_0, \beta, \alpha_0, \alpha_1, B$*, then the drift bound in Theorem 1 holds unchanged.*

Proof. The proof of Lemmas 1–2 is pointwise in the realized candidate family and lower-service table. After conditioning on $Q(t) = Q$ and the scheduler-visible state $\chi(t)$, the same inequality chain gives the same lower-service margin and hence the same quadratic drift bound. Taking conditional expectation over $\chi(t)$ preserves the bound because all constants are uniform over the admitted statewise family.

The Lean 4 theorem prover artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and the statewise version in Corollary 1, where the feasible family $\mathcal{A}_t^{\mathrm{cand}}$ depends on the queue state, available jobs, and measured capacity-boundary rows. It also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds. The paper theorem numbers are not Lean identifiers. Appendix Table 14 gives the exact crosswalk for Theorem 1, Corollary 1, the coordinate-scaled lower confidence bound result, and the operational sandwich boundary. *Scope.* The theorem applies when the stated slack, lower-service, penalty, oracle-error, and moment assumptions are certified; it is not by itself a certificate for every scheduler dispatch.

5.3. Proof decomposition

The preceding lemmas isolate the mathematical obligations that must be certified by the scheduler. Lemma 1 is the capacity and candidate-approximation layer; Lemma 2 is the robust oracle layer; Lemma 3 is the queueing layer; and Theorem 1 is their Foster-Lyapunov composition. The Lean formalization mirrors this separation. The operational necessity direction is kept separate: an "operationally stabilizes" predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from the definition of the predicate.

6. Scheduleurm Instantiation

6.1. Measured action slices

Scheduleurm is an existing scheduler for local and remote research jobs. The new algorithmic layer deliberately separates the theorem-facing policy from the legacy launcher. The theorem-facing

Table 4 Declared measured finite action slices.

Bucket	Dominant service bottleneck	Measured feasible family	Excluded boundary	Use in replay and certificate
q00 light control	control-plane overhead and short-job fragmentation	profiles 1–13	profile 14	replay profile 13; legacy cap 1
q01 GPU-bound compute	accelerator co-location and memory pressure	profiles 1–8	none observed in this node bucket	replay profile 1; profiles 4 and 8 retained for support and LCB certificates; legacy cap 3
q01 CNN ResNet-50	convolutional training throughput and tail drain	profiles 1–7	profile 8 placement/stability boundary	static and rolling replay profile 5; LCB support profile 7; legacy cap 3
q01 LLM DistilGPT-2	transformer forward-pass concurrency and memory pressure	profiles 1–6 and 8	profiles 10–11 boundary rows	replay profile 8; sparse online arrivals drain at profile 1; legacy cap 3
q10 CPU-host bound	host-core and memory-bandwidth contention	profiles 1–9	profile 10	replay profile 8; legacy cap 9
q11 CPU/GPU coupled RL	coupled host and accelerator progress	profiles 1–5	profile 6 thread/placement boundary on node007	replay profile 5; mixed-portfolio support includes profile 5; legacy cap 5

A profile is robust feasible only after measured lower-service and admission checks. The boundary column records the first excluded profile when a boundary was observed. Lower confidence bound (LCB) certificates may retain support profiles that are not selected by the finite-batch replay policy.

policy builds global or statewise candidate families with a queue vector, lower-service vector, penalty units, and robust MaxWeight score. Service profiling, replay, baseline comparison, slack accounting, and oracle-gap checks are implemented outside the legacy scheduler. The live scheduler exposes opt-in hooks: a one-task placement scorer and a global-batch soft-hint path. The global-batch hook computes a bounded candidate configuration and returns only a preferred-node hint to the legacy launcher. Final placement therefore still passes through the existing resource and launch checks. This design lets us test the theorem-facing policy without making it the default production dispatcher.

Table 4 maps each measured workload bucket to the finite action slice used by the theorem-facing policy. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket. The table is therefore both an experimental summary and a definition of the declared finite slices used in the capacity and replay claims.

The replay policy uses a queue-adaptive robust MaxWeight penalty selector rather than a fixed co-location profile. For a workload with backlog count b, it first restricts profiles to a measured support envelope $\mathcal{K}_i$ and then chooses

$$k_i^\star(b) \in \arg\max_{k\in\mathcal{K}_i} \left\{ b\,\widehat{\mu}_i(k) - \theta_{\mathrm{prof}}\widehat{\mu}_i^{\max} k \right\}, \qquad \theta_{\mathrm{prof}} = 0.10. \tag{32}$$

Table 5 Theorem-policy crosswalk in Scheduleurm.

Theorem object	Scheduleurm representation	Interpretation
Queue vector Q	backlog vector recorded for each replay or live slot	Defines the backlog weights in $Q^\top \mu^{\mathrm{L}}(a)$.
Candidate action $a \in \mathcal{A}_t^{\mathrm{cand}}$	finite placement, co-location, admission, and drain-rule configuration	Represents a global or statewise placement/co-location configuration for the theorem-facing oracle.
State index $\chi(t)$	alive nodes, pinned jobs, queued jobs, available-device state, and measured admission status	Selects the statewise candidate family without changing the fixed-family drift constants.
Lower service $\mu^{\mathrm{L}}(a)$	conservative measured-service row	Conservative service vector used in the drift inequality.
Admission and probe state	measured bucket, capacity boundary, lower-confidence status, and probe/defer flag	Determines whether a row enters the theorem-facing population or receives zero theorem service.
Penalty $K_t(a) + G_t(a)$	bounded switching, risk, and delay-control terms	Consumes bounded or queue-scaled slack.
Oracle error (α_0, α_1)	oracle-gap and support-gap reports	Measures approximate maximization loss relative to the theorem objective.
Live placement hook	optional one-task and global-batch scheduler hint	Integration point; theorem use requires candidate, service, penalty, and oracle-gap fields.

The theorem-facing rows expose robust MaxWeight score semantics explicitly; the live hook is an integration surface and not, by itself, a stability certificate.

Here $\mathcal{K}_i$ is the measured support envelope for class i, $\widehat{\mu}_i(k)$ is the measured aggregate service for class i at profile k, and $\widehat{\mu}_i^{\max}$ is the best measured aggregate service in the same envelope. Thus the replay candidate uses lower-interference profiles such as q01 profile 1 when they stay within the measured support slack, while support certificates retain actions such as q01 profiles 4 and 8 for capacity accounting. The optional statewise drain is lower-service guarded and no-upshift: it may only reduce a profile when the measured lower-service and waiting-backlog checks certify no loss relative to the base action. This implementation matches the approximate-oracle theorem: the support envelope preserves the drift action at large Q, and the finite-batch delay preference is a bounded profile penalty that is paid from the same slack budget as $K_t + G_t$.

Table 5 maps the theorem objects to the Scheduleurm implementation. Rows generated by the theorem-facing policy expose the robust MaxWeight objective directly. Rows generated by the live hook require the same candidate, service, penalty, and oracle-gap fields before they are used as theorem evidence.

6.2. Theorem evidence populations

Scheduleurm distinguishes three populations. The explicit task-list replay population is used for performance comparison. The completed-active production population is used for service-map and oracle-gap checks. Raw scheduler history is treated as telemetry because it contains cancelled, attempted-only, benchmark, and externally adopted jobs.

Table 6 Candidate policy versus legacy Scheduleurm on explicit task lists.

Task list	Candidate profile	Makespan	Mean flow	p90 flow
q00 light control	light-control profile 13	11.814×	11.758×	11.814×
q01 GPU compute	JAX GPU profile 1	1.470×	1.560×	1.555×
q01 CNN ResNet-50	CNN profile 5	1.259×	1.173×	1.164×
q01 LLM DistilGPT-2	LLM profile 8	2.723×	2.622×	2.669×
q01 GPU model portfolio	JAX GPU 1; CNN 5; LLM 8	1.613×	1.599×	1.624×
q10 CPU-host bound	CPU profile 8	1.539×	1.535×	1.534×
q11 CPU/GPU coupled	hybrid RL profile 5	1.005×	1.000×	1.000×
Mixed portfolio	CPU 8; JAX GPU 1; CNN 4; LLM 8; hybrid RL 5	1.012×	1.162×	0.994×

Ratios are legacy divided by candidate, so larger is better for Scheduleurm candidate. Each row uses the current robust feasible family; q11 profile 6 and above are excluded. p90 is the 90th percentile of job flow time.

The completed-active Scheduleurm-controlled production population has 3,437 mapped records out of 3,437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service bridge with $\alpha_0 = \alpha_1 = 0$. A longer live-state dry-run covers 128 placement slots and 767 candidate rows without writing the queue or launching tasks. A separate bounded q01/q11 ScheduleurmBench run exercises the robust lower-service hook on real tasks. These checks show that the theorem-facing fields can be generated from the scheduler state. They do not treat raw telemetry as a clean arrival stream.

7. Computational Evidence

7.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. The online-arrival experiment then runs Poisson and bursty traces at load factors 0.50, 0.70, 0.85, and 0.95 with three random seeds for each task set.

Table 6 compares the current candidate policy with the legacy Scheduleurm caps. The metric is completion of all jobs in the task list, not the time to finish a single job. In the task names, convolutional neural network (CNN) denotes the vision workload family and large language model (LLM) denotes the language-model workload family.

Scope. Table 6 compares all-job completion on declared task lists using measured service rows.

Table 7 summarizes the online-arrival experiment. It contains 192 scenarios: eight task sets, two arrival modes, four load factors, and three seeds. The adaptive candidate completed every job in every scenario. Relative to legacy caps, its geometric-mean improvement was 1.618× for

Table 7 Online replay distribution over 192 scenarios.

Metric	Geomean/mean	Median	Min	5%	95%
Legacy / candidate makespan	1.618	1.248	1.003	1.007	9.835
Legacy / candidate mean flow	4.180	2.492	1.000	1.027	179.178
Candidate mean backlog jobs	7.783	8.318	1.099	1.763	27.657
Candidate max backlog jobs	23.654	24.000	4.000	7.000	65.000

No online scenario Pareto-dominated the candidate. No ablated policy Pareto-dominated the full candidate. Some single-metric losses against external-policy diagnostics exceed the 0.5% tolerance, but none is a Pareto-dominance violation.

makespan and 4.180× for mean flow. The worst scenario-level ratios over legacy were 1.003× for makespan and 1.000× for mean flow. No online scenario Pareto-dominated the candidate after the task-native estimated-time-to-completion (ETA) refresh and the trace-context action-union selector. The maximum observed candidate backlog was 90 jobs, and the maximum time-weighted mean backlog was 48.112 jobs.

The ablation experiment isolates the effects of legacy caps, scalar profile selection, support-only makespan scoring, delay-only mean-flow scoring, statewise interference guarding, bounded profile penalties, and the profile tie-break rule. Across the 24 static, Poisson, and bursty traces, no ablation Pareto-dominated the full queue-adaptive robust lower-service scorer. The full candidate retained geometric improvements of 1.736× in makespan and 3.005× in mean flow over legacy. These comparisons use the same measured service cache as the main replay.

7.2. Policy-semantics replay baselines

No single external scheduler covers all four Scheduleurm quadrants and the mixed portfolio without changing the workload model. For this reason the external-scheduler evidence is organized as a diagnostic layer rather than as the central claim of the paper. We compare against policy-semantics replay baselines on the same measured service cache: a throughput-table goodput policy representing Gavel, Pollux, and Sia-like schedulers (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023); a finite-batch delay oracle representing shortest remaining processing time (SRPT), Gittins, and shortest expected remaining processing time (SERPT)-style policies (Scully et al. 2020); and an interference guard representing IADeep/Salus/Gandiva-like co-location control (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Because every row is evaluated through the same measured lower-service map, these baselines test whether the finite candidate family is missing an obvious scheduling semantic; they are not direct binary executions of the external systems.

We also register 11 non-adjacent scheduler families as finite actions in the Scheduleurm+external action union. This is the form needed by the candidate-set theorem: once an external policy family

Table 8 Task-list policy-semantics aggregate on identical measured service cache.

Policy-semantics baseline	Makespan ratio	Mean-flow ratio
Throughput table goodput	1.0068×	1.0003×
Delay oracle	1.0068×	1.0003×
Interference guard	1.0117×	1.0011×
Finish-time fairness	1.00002×	1.00000×
Resource-adaptive goodput	1.00000×	1.00000×
Packing guard	1.00002×	1.00000×
Quadrant composite	1.0068×	1.0003×

Entries are aggregate baseline-versus-candidate ratios across the static and Poisson task-list scenarios, so values above one favor the candidate. A row would dominate the candidate only if both makespan and mean-flow ratios were materially below one; no such row appears under the 0.5% replay tolerance.

is represented by a lower-service row and a bounded penalty, it can be evaluated by the same robust selector. Several systems also have scoped same-host runtime checks, and Decima has a same-domain Spark directed acyclic graph (DAG) simulator check. These checks support traceability. They are not used as the central comparison because the paper's optimization claim is about measured configuration actions, not about deploying every external control plane under identical production conditions.

The multi-node check covers Scheduleurm's own measured history. Two GPU nodes were reachable during the read-only fabric probe. The resulting candidate set contained 18 lower-service rows and 626 disjoint robust-MaxWeight configurations, including a selected two-node configuration with $\alpha_0 = \alpha_1 = 0$. A separate scheduler-history study found 4,058 strict theorem-admitted launches, 4,047 completions, 35 workload domains, 13 nodes, three GPU nodes, and no strict unadmitted launched rows. This supports a Scheduleurm-native multi-node history claim. It does not require claiming that each external scheduler has been run through its original multi-node control plane.

Table 8 reports the task-list policy-semantics aggregate over the 16 static and Poisson measured-cache scenarios used by the external-action-union gate. The 192-scenario online replay is reported separately in Table 7. In both gates, the candidate is not Pareto-dominated by any registered external-policy diagnostic row under the same measured service cache. The expanded q01 CNN/LLM buckets remove the earlier aggregate makespan tradeoff with the throughput-table endpoint: the candidate now improves makespan and mean-flow against that endpoint, although some single-metric scenario losses remain as diagnostic rows rather than Pareto-dominance violations.

We also evaluate the stronger theory-facing construction in which external policy-family actions are not merely baselines but are included in the candidate set. The selector evaluates the Scheduleurm

candidate action together with the throughput-table, delay-oracle, interference-guard, finish-time-fairness, resource-adaptive, packing-guard, and measured bridge-action families under the same measured service units. The action is not deduplicated by co-location profile alone: a candidate is a finite configuration trajectory containing the placement/profile, resource partition, admission order, and statewise drain rule. This distinction matters on the ResNet-50 bucket, where the same profile supports different finish-time and interference-drain trajectories. Table 9 shows that the metric-specific action unions match or improve the external-policy envelopes on every measured task-list scenario within the 0.5% replay tolerance. The fixed Pareto-slack union policy goes one step further for the measured static/Poisson task-list frontier: it enumerates the finite measured Scheduleurm+external action family and chooses the configuration maximizing the smaller of the normalized makespan and mean-flow slack. The online-arrival replay uses the same finite action union with a trace-context online-Pareto-slack tie-break: closed-batch ResNet-50 queues use the measured Scheduleurm tail-drain action, while rolling Poisson or bursty ResNet-50 queues use the measured resource-adaptive admission trajectory already registered in the candidate union; heterogeneous portfolios keep the measured Scheduleurm bridge action. Under these fixed rules, no registered external-policy diagnostic row Pareto-dominates the candidate in the measured-cache task-list frontier or in the 192 online scenarios; the guarded and adaptive-scalarized rows remain useful ablations rather than the main dominance certificate.

Table 9 External-policy action union on identical measured service cache.

Policy family	Sum makespan	Mean flow	Interpretation
Current Scheduleurm online-Pareto-slack candidate	148926.490	3129.563	Theorem-facing fixed candidate.
External best fixed policy	148926.843	3129.567	Sia/Pollux-style resource-adaptive goodput.
Scheduleurm+external Pareto-slack union	148926.972	3129.577	Static/Poisson measured-cache frontier certificate.
Scheduleurm+external makespan union	148962.131	3129.365	Metric-specific action-union diagnostic.
Scheduleurm+external guarded union	149584.066	3129.696	Guarded mean-flow action-union ablation.
Legacy Scheduleurm caps	341169.248	16137.121	Legacy fixed-cap reference.

Times are in seconds. The action-union rows are policy-semantics selectors over measured service actions, not direct executions of Gavel, Pollux, Sia, IADeep, or Salus. The Pareto-slack policy is not Pareto-dominated by any registered external-policy row and matches both measured-cache envelopes within tolerance.

The strict measured-cache claim is separate from a direct-system claim. On the measured cache, every task-list scenario is noninferior to the registered external-policy envelope in both makespan

and mean flow. The last missing row was the ResNet-50 tail/admission case. The final fixed candidate treats a configuration action as a measured trajectory, not merely as a co-location profile: closed-batch ResNet-50 queues use the measured profile-5 tail-drain SRF action, and rolling Poisson/bursty ResNet-50 queues use the measured profile-5 resource-adaptive admission trajectory. This resolves the q01 ResNet-50 row and the inherited q01 portfolio and mixed-portfolio rows. Across q00, q01, q10, q11, and the mixed portfolio, the candidate is noninferior to the measured-cache policy-semantics frontier. The statement remains a measured-cache policy-semantics claim, not a direct full-stack superiority claim over external binaries.

The implementation layer includes the same action-union rule as an optional candidate policy. It adds a Pareto-slack Scheduleurm+external selector, state-dependent marginal service rows, bounded global batch lookahead, reusable online lower-confidence-bound (LCB) estimated time remaining (ETA) estimates, and external-policy action families for finish-time fairness, adaptive goodput, and packing guards. These additions are useful because the measured service curves are not monotone in load. In the controlled slice, a small CUDA task with a high video random-access memory (VRAM) resident job reached 1.095× the empty reference rate, and CPU-resident marginal rows reached 1.001×–1.091× the empty CPU reference. These rows expand the finite candidate family used by the theorem-facing replay; they are not used as evidence that the production scheduler already defaults to the optional global batch policy.

On the q01 GPU-heavy list, the node007 profile-1 measurement now lies on the measured service envelope, so the candidate and the table-goodput/delay external-policy rows all select profile 1 on the single-workload q01 replay. The implemented statewise logic is a lower-service no-upshift drain, not a hidden post-hoc rule: it cannot raise the base action and it only lowers a profile after service and waiting-backlog guards pass. The optional global-batch version improves q01 compute, q01 LLM, the q01 model portfolio, and the mixed portfolio relative to the current candidate. It leaves q01 CNN, q10, and q11 non-regressed under the 0.5% replay tolerance. The paper therefore treats global batching as an algorithmic upgrade path rather than as a default production-dispatch claim.

7.3. Theorem-condition and live-sanity checks

The measured portfolio finite slice also instantiates the slack inequality in Theorem 1. With load $\lambda_i = 0.8\,\mu_i(a_{\text{selected}})$, the candidate family equals the measured action slice, so $L\rho = 0$. The resulting

certificate is

$$\delta = 0.081934686,$$
$$\epsilon_{\text{est}} = \beta = \alpha_1 = 0, \tag{33}$$
$$\eta = 0.081934686.$$

The finite-support second-moment bound is $B = 11209514.9680047$, and the resulting finite-set threshold is $N = 136810386$. The selected measured service vector uses CPU profile 8, JAX GPU profile 1, CNN profile 4, LLM profile 8, and hybrid RL profile 5. This is a theorem-condition certificate for a declared finite-support load model; it is not a claim that the static replay trace is itself a stationary arrival process. The zero modeled losses arise because this certificate uses an exact enumerated measured slice and an exact service-map oracle. They should not be read as evidence that a broad fabric-cover metric has already been calibrated with $L\rho = 0$, or that holdout lower-service error vanishes outside the measured rows. For smaller candidate families, the artifact now reports a greedy finite-feature k-center cover curve; those rows quantify the nonzero $L\rho$ that must be paid before invoking the fabric-cover theorem away from the exact measured slice.

The selected-profile stochastic LCB diagnostic is computed on profile-level aggregate service windows, matching the service coordinate used by the theorem. All 11 selected targets have at least 20 aggregate windows after the supplemental progress measurements. The stronger mean-service statements are not supported by these data: the absolute mean-service holdout diagnostic gives $\epsilon_{\text{est}} = 1196.0765$ and $\eta = -1195.9945$, while the diagonal-normalized diagnostic gives $\epsilon_{\text{est}}^{\text{norm}} = 0.9687$ against normalized slack $\delta^{\text{norm}} = 0.2$, hence $\eta^{\text{norm}} = -0.7687$. The stochastic certificate therefore uses the LCB service vector itself as the lower-service action family. For arrivals scaled to 0.8 of that vector, the selected-profile lower-service capacity certificate has positive slack $\delta_{\text{LCB}} = 0.0341241$. The coordinate LCB ratios for CPU, CNN, JAX GPU, LLM, and hybrid RL are 0.3560, 0.6506, 0.7881, 0.6550, and 0.4165, respectively. Table 10 reports the resulting positive drift margins. *Scope.* The positive η values in Table 10 are finite measured-slice certificates. The stochastic result is a lower-service capacity certificate, not a certificate for 0.8 of the measured mean-service vector.

The replay model was checked against fresh progress-window live measurements on representative measured service-cache rows, not only on the final selected profiles. For the q01 profile-4 row, replay predicted a makespan of 1661.313 seconds, whereas the fresh live proxy was 1652.248 seconds, a relative error of 0.005457. For q11 profile 2, replay predicted 38382.365 seconds and the fresh live proxy was 37760.000 seconds, a relative error of 0.016215. For the mixed portfolio, composed live slices gave relative errors of 0.007349 on makespan, 0.005417 on mean flow, and

Table 10 Finite measured lower-service capacity certificates.

Task set	Selected measured profiles	η
q00 light control	light-control profile 13	123.0121
q01 GPU-bound compute	JAX GPU profile 1	18.3409
q01 CNN ResNet-50	CNN profile 7	13.3236
q01 LLM DistilGPT-2	LLM profile 8	449.7591
q01 GPU model portfolio	JAX GPU 1; CNN 7; LLM 8	13.3236
q10 CPU-host bound	CPU profile 8	10.2331
q11 CPU/GPU coupled	hybrid RL profile 5	0.053191
Mixed portfolio	CPU 8; JAX GPU 1; CNN 7; LLM 8; hybrid RL 5	0.053191

Each row is a stability certificate, not a replay-performance estimate: the finite-slice rows use the finite measured lower-service model at load fraction 0.8, and the selected-profile stochastic row uses the LCB lower-service model described in the text. Performance replay tables elsewhere use empirical measured means on the same service cache. The certificates here support the declared finite slice and do not claim stochastic generalization beyond the measured or LCB-admitted profiles.

0.007964 on p90 flow. These are progress-window sanity checks; the long jobs were not all run to natural completion.

We also ran a corner-case marginal-efficiency experiment while the cluster was idle. Each probe used benchmark logs with explicit progress rate and estimated time remaining (ETA) lines. After three stable windows the probe stopped and the measured service row was available for replay. Table 11 reports the last stable marginal rate and the ratio to the corresponding empty resource baseline. The 30GB row holds a four-GPU resident memory job on the direct node007 GPU host and then adds a small CUDA task on one card. The CPU rows run 96-worker and 180-worker resident jobs on 192-core nodes and then add a small CPU task. These rows show that high resident video memory does not by itself imply a low marginal CUDA rate on this slice; on the idle 192-core CPU node, the single-thread marginal CPU micro-kernel also retained its empty-node rate under the 96-worker and 180-worker resident loads used here. Scheduleurm admits all stable positive-service rows in this controlled slice. Memory-packing policies admit the GPU rows when memory fits, and goodput policies admit rows whose loaded-to-empty ratio exceeds 0.90. The five stable canonical rows also have positive row-level lower-service slack at an offered load of 0.8 times the measured lower-service rate. This makes the corner-case rows theorem-facing for this finite measured slice without changing the default replay cache. Table 12 reports the same rows using job completion time (JCT) rather than marginal service rate.

A controlled live-dispatch check ran q01/q11 ScheduleurmBench tasks through the optional robust lower-service hook. A 6-task run emitted 7 theorem-grade dispatch slots and 13 candidate rows. A larger 32-task run on an available two-GPU node emitted 32 theorem-grade slots and 56

Table 11 Controlled corner-case marginal service probes.

Scenario	Host bucket	Resident rate	Marginal rate	Loaded/empty ratio
CPU empty + small CPU	node001 CPU host	–	17.504	–
CPU 96-worker resident + small CPU	node001 CPU host	33.573	17.354	0.991
CPU 180-worker resident + small CPU	node001 CPU host	20.337	17.383	0.993
GPU empty + small CUDA	node007 GPU host	–	491.827	–
30.0GB GPU resident + small CUDA	node007 GPU host	1049.791	443.983	0.903

Rates are benchmark service units per second from explicit progress logs. The loaded/empty ratio compares the marginal row with the same small-task empty-resource baseline. The table is a controlled synthetic marginal-service slice, not a full-stack state-of-the-art performance claim or a production-wide theorem population.

Table 12 Gavel-style resident-delay/JCT holdout on the controlled co-location rows.

Scenario	Co-run mean JCT	Defer mean JCT	Mean JCT reduction	Makespan reduction
CPU 96-worker resident + small CPU	0.044	0.053	17.0%	29.1%
CPU 180-worker resident + small CPU	0.053	0.054	1.4%	30.4%
30.0GB GPU resident + small CUDA	0.217	0.322	32.5%	46.8%

Seconds are measured benchmark service-window seconds. The defer baseline uses the fastest observed resident-alone progress window, which favors defer. The table is a scoped measured-service holdout, not a direct Gavel full-stack execution.

candidate rows, completed all 32 tasks, and had $\alpha_0 = \alpha_1 = 0$. These runs check that the hook can launch and complete admitted tasks. They are separate from the larger production-history evidence.

The production evidence is based on strict scheduler history rather than on raw telemetry. Rows outside scheduler control, including attempted-only, auto-adopted, diagnostic, and read-only probe rows, are excluded. The admitted history contains 4,058 strict theorem-facing launches, 4,047 completions, 35 workload domains, 13 nodes, and no strict unadmitted launched rows. For closed shadow-subset snapshots, the theorem bridge has $\alpha_0 = \alpha_1 = 0$; CPU fallback slots use legacy sort-key semantics and are excluded from the theorem subset. The live evidence therefore supports controlled completion, strict scheduler-history completion, and read-only multi-node Scheduleurm coverage. It does not convert raw telemetry or future unknown jobs into theorem-facing arrivals. Table 13 summarizes the four evidence populations.

7.4. Formal and production artifacts

The Lean proof artifact is submitted as a supplement. It contains the consolidated proof file, the Lean toolchain specification, the build log, a paper-theorem to Lean-theorem crosswalk, and checksums for the submitted files. The supplement build passes, and a comment-aware proof search finds no `sorry`, `admit`, or `axiom` proof terms. Full file hashes are reported in the supplement manifest rather than in the manuscript text.

Table 13 Controlled and production evidence populations.

Population	Evidence	Interpretation
Controlled 6-task launch	6 launched and completed tasks; 7 theorem slots; 13 candidate rows.	Hook-level launch/completion check for admitted candidate rows.
Controlled 32-task launch	32 launched and completed tasks; 32 theorem slots; 56 candidate rows; $\alpha_0 = \alpha_1 = 0$.	Larger controlled live check on a two-GPU node.
Strict scheduler-history production	4,058 admitted launches; 4,047 completions; 35 domains; 13 nodes; no strict unadmitted launched rows.	Production-history evidence for admitted rows; raw 30-day telemetry is not the theorem population.
Completed-active service-map bridge	3,437 strict mapped records out of 3,437; mapped-slice slack 9.123×10^{-6}; service-map oracle bridge with $\alpha_0 = \alpha_1 = 0$.	Lower-service bridge for the controlled completed-active population.
Rolling queue snapshots	Active-production progress and closed shadow-subset oracle checks when available; CPU fallback slots excluded from the theorem subset.	Read-only monitoring and oracle conversion, not raw-history closure.
Live-oracle boundary	Large-scale live-oracle trace fields and global-batch soft-hint hook exist, but production-default global dispatch is not claimed.	Separates hook readiness from deployed theorem-grade production scheduling.

The populations are kept separate because they support different claims: controlled live execution, strict admitted-history completion, service-map bridging, read-only monitoring, and live-oracle readiness.

For production data, the current theorem-facing population is the completed-active Scheduleurm-controlled set. It has 3,437 strict mapped records out of 3,437, with mapped-slice slack 9.123×10^{-6}. The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. By contrast, the raw 30-day history has 6,957 records, with 4,825 mapped records and 2,132 unmapped records, so it is not a clean theorem population. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw, attempted-only, external, or future scheduler row.

8. Discussion and Limitations

This paper connects a queueing-stability theorem with the objects available in a real heterogeneous scheduler. The main mathematical point is that candidate restriction is not treated as a heuristic shortcut. It is represented as an explicit support-function loss and paid from capacity slack together with estimation, penalty, and oracle losses. This gives a readable accounting rule: a scheduler may use a finite candidate family only when the remaining drift margin is still positive.

The Scheduleurm case study shows how this accounting can be implemented. The scheduler records measured lower-service rows, admissible configuration actions, capacity boundaries, penalties, oracle gaps, and admission status. The same records support four roles: performance replay, stability-condition calibration, production-history classification, and optional live-hook checks. This separation is useful because it prevents a strong empirical result in one population from being silently transferred to another.

The strongest empirical statement remains a measured finite-slice statement. For the declared service domains, exact measured actions give $\rho = 0$, and compressed candidate families report the

corresponding nonzero $L\rho$. A future task is theorem-facing only if it is admitted to a measured domain with a positive lower-service row; otherwise it is assigned to a probe/defer state with zero theorem service. This is sufficient for the present measured-domain certificate, but it is not a universal positive-service theorem for all future workloads and hardware states.

The external-scheduler comparisons have a similarly precise role. They test whether the measured candidate family omits a useful scheduling semantic, such as goodput allocation, delay priority, or interference guarding. Once such a semantic is represented by a lower-service row and a bounded penalty, it becomes another finite action in the robust selector. This is the right comparison for the candidate-set theorem. It is narrower than a claim that Scheduleurm has beaten every external scheduler in its native full-stack deployment.

The production evidence should also be read at the correct level. Controlled live launches show that admitted theorem-hook actions can run to completion. Strict scheduler history shows that admitted production rows can be classified and completed at scale. Read-only shadow traces show that current queue states can be converted into theorem-facing candidate rows without mutating the queue. Together these checks support the measured Scheduleurm certificate. They do not turn raw telemetry, future unknown arrivals, or every live dispatch into a stability-certified arrival stream.

Two technical directions remain open. The first is broader fabric-cover calibration: extending the positive-service certificate beyond the declared finite domains requires perturbation evidence for the feature map, projection, cover radius, Lipschitz envelope, and statistical failure probability. The second is online learning: the active-bucket sampler and regime detector have formal conditional guarantees and replay checks, but they should be validated in live A/B experiments before becoming default scheduling logic.

9. Conclusion

We studied heterogeneous compute scheduling as a configuration-action stochastic processing network. The resulting robust candidate MaxWeight theorem gives an explicit slack budget for candidate approximation, lower-service estimation, bounded penalties, and approximate optimization. If the residual margin is positive, the policy satisfies a finite-set Foster-Lyapunov drift certificate. The fixed-family and statewise statements are formalized in Lean.

Scheduleurm provides a concrete instantiation of the theorem. Its measured finite slices close the lower-service stability conditions for declared workload domains. On explicit task lists and 192 online replay scenarios, the robust candidate improves legacy completion metrics and is not

Pareto-dominated by registered policy-semantics baselines evaluated on the same measured service cache. Controlled live launches and strict scheduler-history evidence show how the theorem-facing objects can be produced by an operational scheduler.

The contribution is therefore not a universal claim about every future workload or every external scheduler implementation. It is a mathematical and empirical template for making approximate configuration-action scheduling certifiable: define the action family, measure conservative service, account for every loss, and claim stability only when the residual slack remains positive.

Appendix: Conventional Proof Details

This appendix gives the conventional, paper-readable proof chain behind Theorem 1. It is intentionally redundant with the main text: the purpose is to make clear which part is mathematics, which part is a measured Scheduleurm certificate, and which part is checked by Lean. The Lean artifact is a formal check of the algebraic and recurrence implications; it is not a substitute for empirical certificates of service rows, lower confidence bounds, candidate admission, oracle gaps, or moment bounds.

EC.1. Capacity Slack as a Support-Function Margin

Let $\mu(a) \in \mathbb{R}_+^{|\mathcal{I}|}$ denote the true mean service vector of full action $a \in \mathcal{A}^{\text{full}}$, and let $H_{\mathcal{A}^{\text{full}}}(q) = \max_{a\in\mathcal{A}^{\text{full}}} q^\top \mu(a)$ be the full-action support function for nonnegative backlog weights $q \in \mathbb{R}_+^{|\mathcal{I}|}$. A stationary randomized policy over full actions is a probability vector $x \in \text{Mix}(\mathcal{A}^{\text{full}})$. If it has coordinate slack $\delta > 0$ against the arrival mean λ, then

$$\lambda_i + \delta \le \sum_{a\in\mathcal{A}^{\text{full}}} x_a \mu_i(a), \qquad i \in \mathcal{I}. \tag{34}$$

Multiplying each coordinate by $q_i \ge 0$ and summing gives

$$q^\top \lambda + \delta \|q\|_1 \le \sum_{a\in\mathcal{A}^{\text{full}}} x_a q^\top \mu(a). \tag{35}$$

Because x is a convex combination, the right-hand side is at most the maximum full-action support:

$$\sum_{a\in\mathcal{A}^{\text{full}}} x_a q^\top \mu(a) \le H_{\mathcal{A}^{\text{full}}}(q). \tag{36}$$

Hence full-action capacity slack implies

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q). \tag{37}$$

This is the only place where the capacity-region hypothesis enters the drift proof. It converts an interior capacity statement into a MaxWeight support margin that can be compared to candidate actions. The paper uses the support-function statement as Assumption 1; the operational necessity direction is separate and requires a load-certified stochastic model and a conservation law.

EC.2. Candidate Fabric Cover and Support Loss

The scheduler cannot enumerate every full configuration. It evaluates an admitted candidate family $\mathcal{A}_t^{\mathrm{cand}} \subseteq \mathcal{A}^{\mathrm{full}}$, possibly depending on the observed state t. The finite-candidate theorem needs a support loss certificate: for every $q \geq 0$,

$$H_{\mathcal{A}^{\mathrm{full}}}(q) \leq H_{\mathcal{A}_t^{\mathrm{cand}}}(q) + \epsilon_{\mathrm{cand}} \|q\|_1 . \tag{38}$$

When the loss is obtained from a calibrated fabric metric, let $\Phi(a)$ be the finite feature representation of action a, let d_Φ be the feature metric, and suppose each full action a has a projected candidate $\pi_t(a) \in \mathcal{A}_t^{\mathrm{cand}}$ satisfying

$$d_\Phi(a, \pi_t(a)) \leq \rho. \tag{39}$$

If service is Lipschitz in this representation,

$$\|\mu(a) - \mu(\pi_t(a))\|_\infty \leq L\, d_\Phi(a, \pi_t(a)), \tag{40}$$

then for any maximizer $a^\star \in \arg\max_{a \in \mathcal{A}^{\mathrm{full}}} q^\top \mu(a)$,

$$\begin{aligned} H_{\mathcal{A}^{\mathrm{full}}}(q) &= q^\top \mu(a^\star) \\ &\leq q^\top \mu(\pi_t(a^\star)) + \|q\|_1 L\rho \\ &\leq H_{\mathcal{A}_t^{\mathrm{cand}}}(q) + L\rho \|q\|_1 . \end{aligned} \tag{41}$$

Thus $\epsilon_{\mathrm{cand}} = L\rho$ in the fabric-cover case. In measured finite slices where the candidate support is certified directly, the artifact uses the observed support gap itself as ϵ_{cand}; when the candidate set is exact on the measured slice, this term is zero. This is why the proof does not shrink the action space informally: every restriction must be paid for by the explicit support-function loss in (38).

EC.3. Lower-Service Rows, Penalties, and Approximate Oracles

The robust scheduler optimizes conservative service, not the unknown true mean service. For each admitted candidate $a \in \mathcal{A}_t^{\mathrm{cand}}$, the artifact provides a lower-service vector $\mu_t^{\mathrm{L}}(a)$. The lower-service support loss is

$$H_{\mathcal{A}_t^{\mathrm{cand}}}(q) \leq \max_{a \in \mathcal{A}_t^{\mathrm{cand}}} q^\top \mu_t^{\mathrm{L}}(a) + \epsilon_{\mathrm{est}} \|q\|_1 . \tag{42}$$

The scheduling score is

$$\Psi_t(a) = Q^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a), \tag{43}$$

where $K_t(a)$ and $G_t(a)$ are bounded switching, risk, or guard penalties. Equation (42) is the lower-confidence-bound bridge from measured service rows to the drift proof. The selected action a_t need not be the exact maximizer. It satisfies

$$\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \Psi_t(a) - \Psi_t(a_t) \le \alpha_0 + \alpha_1 \|Q\|_1 , \tag{44}$$

and all admitted candidate penalties obey

$$0 \le K_t(a) + G_t(a) \le P_0 + \beta \|Q\|_1 , \qquad a \in \mathcal{A}_t^{\mathrm{cand}}. \tag{45}$$

Combining the capacity support bound (37), the two support-loss bounds (38) and (42), the oracle gap (44), and the penalty bound (45) yields the selected lower-service margin. First,

$$\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} Q^\top \mu_t^{\mathrm{L}}(a) \ge Q^\top \lambda + (\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}}) \|Q\|_1 . \tag{46}$$

Second, by the oracle gap and the all-candidate penalty bound,

$$\begin{aligned} Q^\top \mu_t^{\mathrm{L}}(a_t) &\ge \max_{a \in \mathcal{A}_t^{\mathrm{cand}}} Q^\top \mu_t^{\mathrm{L}}(a) \\ &\quad - (P_0 + \alpha_0) - (\beta + \alpha_1) \|Q\|_1 \\ &\ge Q^\top \lambda + \eta \|Q\|_1 - (P_0 + \alpha_0), \end{aligned} \tag{47}$$

where

$$\eta = \delta - (\epsilon_{\mathrm{cand}} + \epsilon_{\mathrm{est}} + \beta + \alpha_1). \tag{48}$$

The theorem therefore has a simple accounting rule: candidate loss, lower-service estimation loss, queue-scaled penalty, and queue-scaled oracle loss all consume the same capacity slack. A stability certificate requires $\eta > 0$. Additive terms P_0 and α_0 do not change the slope of the drift; they enlarge the finite set outside which the drift is negative.

EC.4. Quadratic Drift and the Foster Certificate

The queue update is

$$Q_i(t+1) = \left[Q_i(t) - S_i(t)\right]^+ + A_i(t), \tag{49}$$

where $A_i(t)$ is arrival and $S_i(t)$ is realized service for class i. For $V(Q) = \frac{1}{2}\sum_i Q_i^2$, the standard one-step inequality gives

$$\begin{aligned}\Delta V(t) \le \frac{1}{2}\sum_i \left(A_i(t)^2 + S_i(t)^2\right) \\ + Q(t)^\top A(t) - Q(t)^\top S(t).\end{aligned} \tag{50}$$

Assumption 4 supplies the conditional second-order bound

$$\mathbb{E}\left[\frac{1}{2}\sum_i \left(A_i(t)^2 + S_i(t)^2\right) \middle| Q(t) = Q\right] \le B. \tag{51}$$

It also ties the stochastic process to the theorem-facing service rows:

$$\mathbb{E}[A(t) \mid Q(t) = Q] \le \lambda, \tag{52}$$

$$\mathbb{E}[S(t) \mid Q(t) = Q] \ge \mu_t^{\mathrm{L}}(a_t) \tag{53}$$

coordinatewise. Taking conditional expectation in (50) and using (47),

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\le B + Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t) \\ &\le B + P_0 + \alpha_0 - \eta \left\|Q\right\|_1 .\end{aligned} \tag{54}$$

If N and $\gamma > 0$ satisfy

$$B + P_0 + \alpha_0 + \gamma \le \eta N, \tag{55}$$

then whenever $\|Q\|_1 > N$, (54) implies

$$\mathbb{E}[\Delta V(t) \mid Q(t) = Q] \le -\gamma. \tag{56}$$

This is the finite-set drift certificate used in Theorem 1. To translate the certificate into positive recurrence for a Markov chain, the paper also assumes a countable queue state space, finite sublevel sets of V, and a closed communicating class or equivalent local-return condition. Without those recurrence-side conditions the theorem should be read as a negative-drift certificate, not as an unconditional positive-recurrence statement for every possible implementation.

EC.5. Statewise Feasible Families and Operational Boundaries

The statewise corollary is a conditioning argument rather than a new drift identity. Let $\chi(t)$ record scheduler-visible information such as alive nodes, pinned tasks, queued jobs, and measured admission status. If the candidate family $\mathcal{A}^{\mathrm{cand}}(Q(t),\chi(t))$, lower-service table $\mu^{\mathrm{L}}(Q(t),\chi(t),\cdot)$, penalty bounds, oracle bounds, and moment bounds satisfy the same constants for every admitted $(Q(t),\chi(t))$, then the proof in EC.1–EC.4 holds after conditioning on $(Q(t),\chi(t))$. Taking expectation over $\chi(t)$ preserves the drift bound because the constants are uniform over the admitted family.

The operational capacity statement is kept separate. Sufficiency follows from the positive-slack drift chain above once a stochastic model certifies the arrival mean, service accounting, lower-service domination, moment bounds, and local recurrence conditions. Necessity is not obtained by defining a model to "stabilize" a load; it requires the model to encode the load and obey a conservation law showing that long-run departures cannot exceed the capacity base. This separation is essential for the paper's claim boundary: Lean checks the implication from certified assumptions to drift and recurrence, while the experiments and admission checks certify when Scheduleurm rows satisfy those assumptions.

EC.6. Lean Crosswalk and Artifact Assumptions

Tables 14 and 15 separate the formal proof contract from the empirical Scheduleurm certificates. The manuscript uses paper-facing statement names; exact Lean identifiers are kept in the supplementary theorem crosswalk so that the article does not become a code index.

Table 14 Paper statements and Lean proof groups.

Paper statement	Lean proof group	What the formal result checks
Theorem 1, exact candidate version	Fixed-family exact-oracle robust candidate MaxWeight theorem	Support-function candidate loss, robust lower-service drift, and finite-set recurrence certificate.
Theorem 1, approximate-oracle version	Fixed-family approximate-oracle robust candidate MaxWeight theorem	Calibrated fabric cover, bounded second moment, additive and queue-scaled oracle loss, and slack accounting.
Corollary 1	State-indexed candidate-family theorem	State-dependent feasible families under the same uniform constants used in the fixed-family theorem.
Coordinate-scaled lower-service certificate	Diagonal-scaled lower confidence bound theorem	Heterogeneous service-unit scaling and lower-service support loss in normalized coordinates.
Operational capacity boundary	Load-certified capacity sandwich theorem	Positive-slack sufficiency and zero-slack conservation-law necessity once the stochastic model encodes arrivals and service accounting.

The exact Lean theorem identifiers, file hashes, and build log are supplied in the supplementary artifact crosswalk. This table records the mathematical correspondence used by the paper.

Table 15 Formal hypotheses and Scheduleurm certificates.

Object	Formal role	Scheduleurm certificate or boundary
λ, δ	Arrival mean and full-action capacity slack.	Online arrival replay, load sweep, and finite lower-service capacity checks.
ϵ_{cand} or $L\rho$	Candidate-set or fabric-cover support loss.	Measured finite-slice support gaps, declared bucket admission, and fabric-cover calibration rows when available.
μ_t^{L}, ϵ_{est}	Conservative lower-service rows and estimation loss.	Holdout lower confidence bound checks, measured service cache, and admission/probe status.
P_0, β	Bounded and queue-scaled penalty consumption.	Penalty-unit records, guard/tie-break checks, and bounded switching configuration.
α_0, α_1	Additive and queue-scaled approximate oracle loss.	Oracle-gap check comparing selected action against theorem-facing candidate families.
B, N, γ	Second-order moment bound and finite-set drift threshold.	Workload bounds, controlled trace statistics, and recurrence-side finite-state admission conditions.
Statewise $\chi(t)$	Scheduler-visible feasible-family index.	Alive node, pinned-job, queued-job, service-cache, and admission-state trace fields.
Operational model certificate	Links stochastic arrivals and services to the queueing theorem.	Load-certified model contract plus conservation-law boundary; not inferred from Lean alone.

The table is deliberately conservative: Lean verifies the implications once these hypotheses hold, while the Scheduleurm artifact determines which measured rows satisfy them.

Appendix: Supplementary Evidence Index

The full machine-readable status dashboard is provided in the supplementary artifact package. Tables 16 and 17 give a compact index of what is included and what each evidence family is allowed to support. They summarize the detailed supplementary status dashboard in manuscript-facing evidence categories; the row-level dashboard remains in the supplement, so the compression changes presentation rather than the underlying evidence.

Table 16 Supplementary evidence index: theorem and Scheduleurm certificates.

Gate family	Retained dashboard evidence	Boundary kept in manuscript
Formal proof package	Consolidated Lean upload, toolchain, build log, theorem crosswalk, no-`sorry/admit/axiom` proof search, and checksums.	Formalizes theorem implications only after the stated service, moment, slack, and recurrence assumptions are certified.
Declared finite-domain cover	q00/q01/q10/q11 service-cache summaries, admission boundaries, measured support gaps, all-state safety cover, and probe/defer status for unknown states.	Supports declared finite measured domains; it does not assert positive service for arbitrary future states.
Lower-service and slack certificates	Holdout lower confidence bound (LCB) rows, selected-profile stochastic LCB certificate, row-level lower-service slack, $L\rho$ accounting, and finite second-moment thresholds.	Certifies lower-service capacity for admitted rows; mean-service and broad fabric-cover claims require additional calibration.
Replay and online stress	Explicit task-list replay, Poisson and bursty load sweeps, ablations, guarded-policy comparisons, and slack accounting tables.	Compares policies on the same measured service cache; it is not a proof that a static replay trace is a stationary arrival process.
Controlled launch/completion	Six-task and 32-task theorem-admitted launches, completion records, candidate rows, and $\alpha_0 = \alpha_1 = 0$ oracle-gap checks.	Supports controlled hook execution; it is separate from production-wide organic dispatch claims.
Production-history bridge	Strict completed-active production classification, strict launched/completed counts, read-only shadow traces, service-map oracle bridge, and exclusion of attempted-only or external rows.	Supports admitted history evidence; raw telemetry and future unknown jobs are outside the theorem population.
Live-oracle and global-dispatch readiness	Optional global-batch soft-hint hook, live-oracle trace fields, active-production shadow rows, and production readiness bridge.	Shows hook readiness and read-only conversion to theorem rows; it does not make the global batch policy the default production dispatcher.

This table indexes evidence used for the main theorem, measured finite-slice claims, and Scheduleurm production-history bridge.

Table 17 Supplementary evidence index: external-policy and extension checks.

Gate family	Retained dashboard evidence	Boundary kept in manuscript
Policy-semantics baseline universe	Gavel-, Pollux-, Sia-, IADeep-, Salus-, fairness-, packing-, short-job, and delay-style policy rows evaluated on the same measured service cache.	Tests whether the candidate family omits useful finite scheduling semantics; it is not direct binary/full-stack superiority.
Gavel calibration and holdout rows	Adapter certificate, profile-aware calibration, resident-delay/JCT holdout, native same-workload simulator row, and same-host physical probe.	Keeps scalar service-unit equivalence separate from measured-cache policy comparison and scoped runtime evidence.
Named external runtime probes	Same-host runtime or adapter probes for Gavel, Pollux/AdaptDL, Sia, IADeep, and Salus, with paired workload and environment metadata when available.	Diagnostic implementation evidence only; original multi-node control-plane superiority is not claimed.
External action-union and algorithm upgrades	External policy-family actions, trajectory/action-semantics union rows, adaptive scalarized replay, state-dependent marginal lookup, global lookahead, and ETA reuse.	Shows the candidate family can absorb registered semantics; it is not production-default launch evidence.
Adjacent-domain scheduler bridge	Decima-style Spark-DAG simulator bridge, registered non-GPU scheduling diagnostics, and same-domain simulator notes.	Shows adjacent-domain coverage; it is not a GPU co-location full-stack comparison.
Learning and hidden-regime extensions	Active-bucket concentration, forced exploration, dwell/switching, detector-delay, optional live-hook records, and regime-detector fields.	Extension evidence; the current default live scheduler does not claim a complete online-learning theorem.
Non-future closure and future-state safety	Conservative all-state safety cover, probe/defer admission, zero theorem service for unknown states, and non-future closure rows excluding arbitrary future workloads.	Prevents overclaiming positive service or superiority for arbitrary future workloads, unregistered schedulers, or unmeasured hardware states.

This table keeps external scheduler and extension evidence visible while preserving the paper's scoped mathematical claim.

Code and Data Availability

The code, experiment artifacts, measured service-cache summaries, reviewer supplement manifest, and README files for this submission are provided in the supplementary Scheduleurm artifact package. The package contains the Scheduleurm source tree, theorem-facing replay modules, measured service-cache files, generated experiment summaries, and the Lean supplement headed by `ScheduleurmUpload.lean`. The reproduction script rebuilds the paper PDF, reruns the safe replay and certificate checks, executes the targeted tests, and refreshes the reproduction manifest. Live launches and external full-stack runtime probes are not executed by default; their recorded outputs are packaged with the corresponding environment notes. Raw scheduler history records that are outside the theorem-facing completed-active population are retained as operational telemetry and are not used as the claimed replication population. The Lean supplement package contains `ScheduleurmUpload.lean`, `lakefile.toml`, `lean-toolchain`, `build.log`, `theorem_crosswalk.md`, `manifest.json`, and checksums.

References

Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL `http://dx.doi.org/10.1007/s11134-006-7586-8`.

Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL `http://dx.doi.org/10.1017/S0269964806060335`.

Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL http://dx.doi.org/10.14778/3685800.3685845.

Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM), URL http://dx.doi.org/10.1145/3581784.3607060.

Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL http://dx.doi.org/10.1287/stsy.2021.0081.

Dai JG, Meyn SP (1995) Stability and convergence of moments for multiclass queueing networks via fluid limit models. *IEEE Transactions on Automatic Control* 40(11):1889–1904, URL http://dx.doi.org/10.1109/9.471210.

de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, 625–635 (Springer), URL http://dx.doi.org/10.1007/978-3-030-79876-5_37.

Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL http://dx.doi.org/10.1287/stsy.2021.0087.

Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL http://dx.doi.org/10.1145/3342195.3387547.

Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL http://dx.doi.org/10.1145/3529375.

Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL https://www.usenix.org/conference/nsdi20/presentation/mahajan.

Mao H, Schwarzkopf M, Venkatakrishnan SB, Meng Z, Alizadeh M (2019) Learning scheduling algorithms for data processing clusters. *Proceedings of the ACM Special Interest Group on Data Communication*, 270–288 (ACM), URL http://dx.doi.org/10.1145/3341302.3342080.

Meyn SP, Tweedie RL (2009) *Markov Chains and Stochastic Stability* (Cambridge University Press), 2 edition, URL http://dx.doi.org/10.1017/CBO9780511626630.

Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak.

Neely MJ (2010) *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Synthesis Lectures on Communication Networks (Morgan & Claypool Publishers), URL http://dx.doi.org/10.2200/S00271ED1V01Y201006CNT007.

Peng Y, Bao Y, Chen Y, Wu C, Meng C, Lin W (2019) DL2: A deep learning-driven scheduler for deep learning clusters. *arXiv preprint arXiv:1909.06040* URL https://arxiv.org/abs/1909.06040.

Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL https://www.usenix.org/conference/osdi21/presentation/qiao.

Scully Z, Harchol-Balter M, Scheller-Wolf A (2020) Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4(1):1–29, URL http://dx.doi.org/10.1145/3379477.

Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL http://dx.doi.org/10.1214/aoap/1075828046.

Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL http://dx.doi.org/10.1145/3600006.3613175.

Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL http://dx.doi.org/10.1109/9.182479.

Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL https://www.usenix.org/conference/osdi18/presentation/xiao.

Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL https://proceedings.mlr.press/v206/yang23d.html.

Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcca59228f2-Abstract.html.